



بسمه تعالی

آزمایشگاه معماری کامپیوتر

دکتر سربازی آزاد

نرگس کاری

سروش داوران

گزارش شماره ۵

تاریخ: ۱۲ خرداد ۱۴۰۵

شماره دانشجویی: ۴۰۲۱۱۰۸۲۱

شماره دانشجویی: ۴۰۲۱۰۵۹۸۷

هدف آزمایش

طی این آزمایش (بخش اول از طراحی ماشین پایه)، هدف طراحی و پیاده‌سازی کامل یک واحد محاسبات (ALU) و مجموعه ثبات‌های عمومی یک کامپیوتر ساده است. در این معماری، امکان انجام عملیات جمع و تفریق با انتخاب ثبات‌های مبدأ و ثبات نگه‌دارنده نتیجه (مقصد) فراهم می‌شود تا در نهایت قابلیت اجرای فرمان‌های شش بیتی روی ثبات‌های ۸ بیتی ایجاد گردد.

شرح معماری و تئوری آزمایش

بر اساس معماری خواسته‌شده، ماشین دارای چهار ثبات عمومی ۸ بیتی (R^0, R^1, R^2, R^3) است. یکی از عملوندهای واحد محاسبه همواره به صورت ثابت محتوای ثبات R^0 است و عملوند دیگر می‌تواند یکی از ثبات‌های R^0 تا R^3 یا مقادیر ثابت ۰، ۱ و -۱ باشد. حاصل تولید شده توسط ALU (جمع یا تفریق) به یکی از ثبات‌های مقصد (R^0 تا R^3) منتقل می‌شود.

فرمان‌های این معماری دارای یک قالب ۶ بیتی هستند:

- **بیت ارزش بالا (بیت ۵):** تعیین‌کننده نوع عملیات ALU (مقدار ۰ برای Add و مقدار ۱ برای Subtract).

- **بیت‌های ۳ و ۴ (Destination):** مشخص‌کننده ثبات مقصد برای ذخیره نتیجه (۰۰ تا ۱۱ به ترتیب برای R^0 تا R^3).

- **بیت‌های ۰ تا ۲ (Source):** مشخص‌کننده عملوند دوم (۰۰۰ تا ۰۱۱ برای ثبات‌ها و ۱۰۰ تا ۱۱۰ برای مقادیر ثابت).

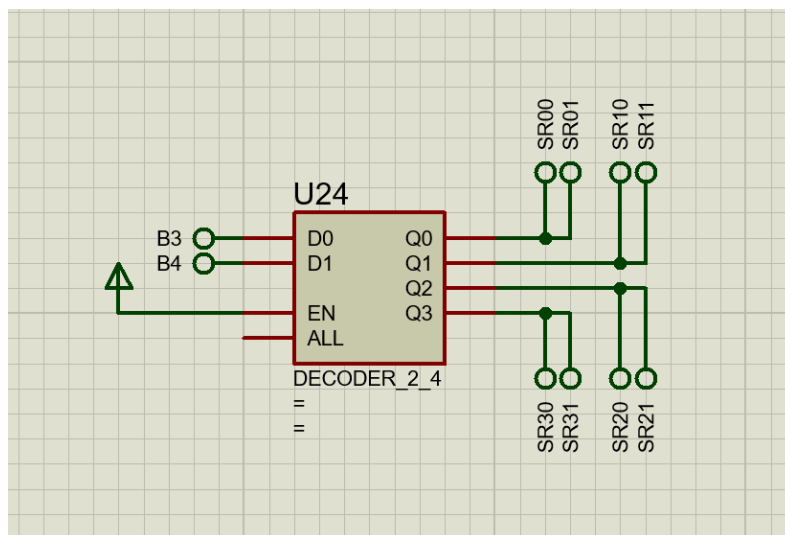
نحوه پیاده‌سازی و جزئیات طراحی

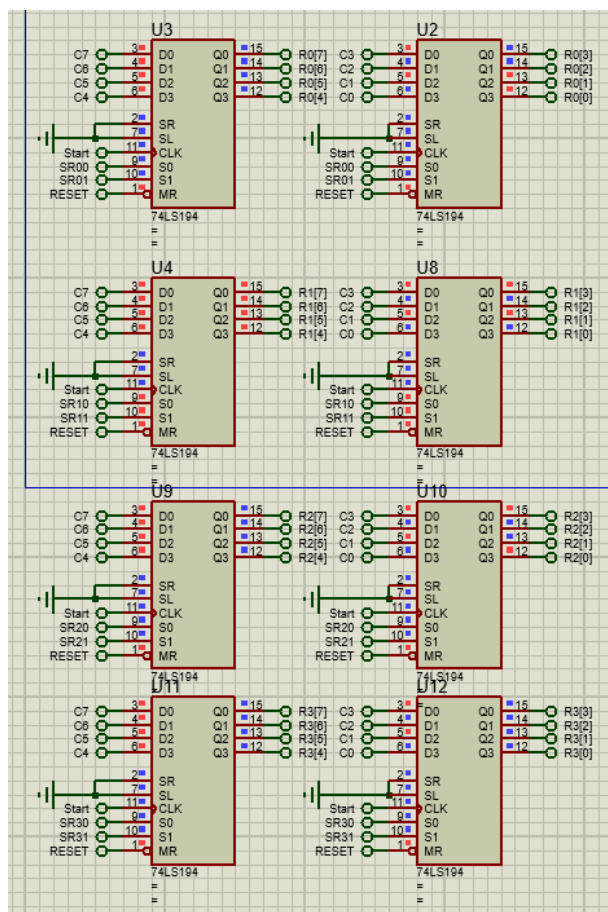
برای پیاده‌سازی این مدار، از رویکرد ماژولار استفاده شده است. جزئیات طراحی هر بخش به شرح زیر است:

نگهداری داده‌ها (ثبات‌ها) و منطق انتخاب مقصد

برای نگهداری مقادیر ثبات‌ها، از **شیفت‌رجیسترها** (Shift Registers) با قابلیت بارگذاری موازی (Parallel Load) استفاده شده است. با توجه به ۸ بیتی بودن داده‌ها، از کنار هم قرار دادن شیفت‌رجیسترهای استاندارد برای تشکیل ثبات‌های ۸ بیتی بهره گرفته‌ایم.

برای کنترل اینکه نتیجه نهایی در کدام ثبات نوشته شود، از یک دیکدر ۲ به ۴ (Decoder) استفاده شده است. ورودی این دیکدر، ۲ بیت مربوط به Destination در قالب فرمان است. خروجی فعال دیکدر، مستقیماً به پایه فعال‌ساز بارگذاری موازی (Parallel Load Enable) ثبات مربوطه متصل می‌شود. برای اعمال و ثبت داده‌ها، از یک ورودی دستی به عنوان کلاک (Clock) استفاده شده است. با تغییر وضعیت این ورودی به '۱'، لبه کلاک شبیه‌سازی شده و داده جدید در ثباتی که توسط دیکدر انتخاب شده است، نوشته می‌شود.

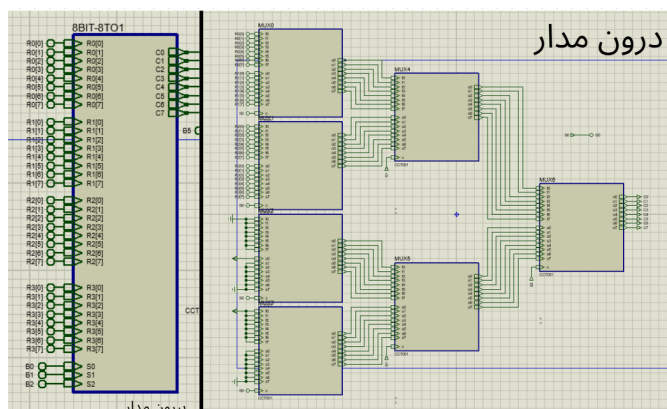




شکل ۱: معماری ثابت‌ها و استفاده از دیکدر برای فعال‌سازی سیگنال Load

انتخاب عملوند مبدأ (Source)

برای انتخاب عملوند دوم واحد محاسبات، از یک **مالتی‌پلکسر ۸ به ۱** (Multiplexer) استفاده شده است. پایه‌های کنترلی (Select) این مالتی‌پلکسر به ۳ بیت مربوط به Source در قالب فرمان متصل شده‌اند. ورودی‌های این مالتی‌پلکسر شامل خروجی ثابت‌های R_0 تا R_3 و مقادیر ثابت ۰، ۱ و -۱ (به صورت باینری ۸ بیتی یعنی 11111111) می‌باشند.



شکل ۲: مدار مالتی‌پلکسر برای انتخاب عملوند دوم

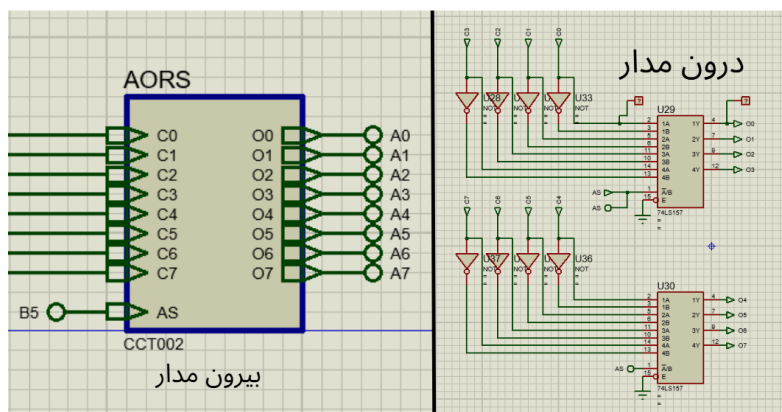
واحد محاسبات (جمع و تفریق)

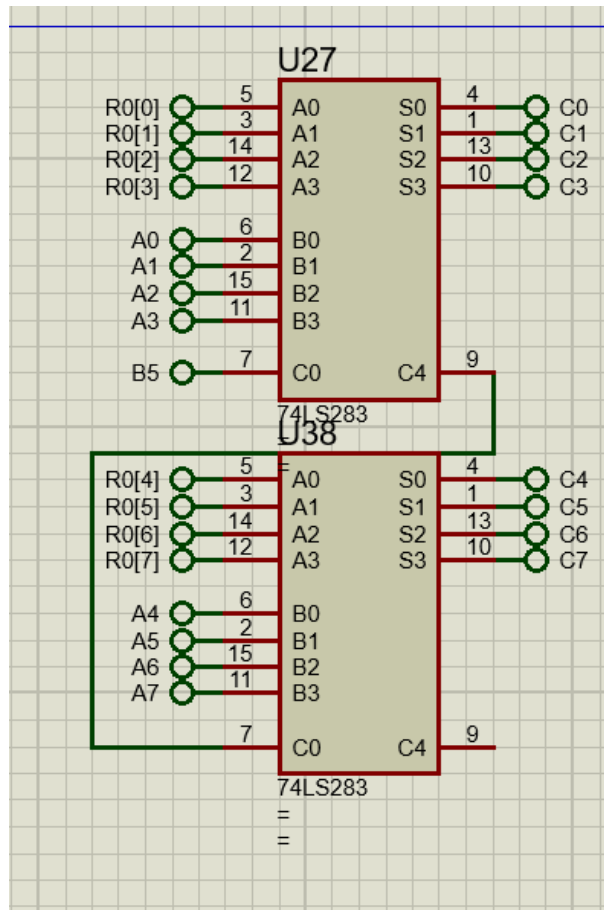
منجز پردازشی این مدار یک جمع کننده ۸ بیتی است. یکی از ورودی‌های این جمع کننده همواره مستقیماً از خروجی ثابت R_0 تأمین می‌شود. برای پیاده‌سازی عملیات تفریق بدون نیاز به استفاده از یک آی‌سی تفریق‌گر مجزا، از منطق مکمل دو (Two's Complement) استفاده شده است:

۱. **زیرمدار مکمل گیر (NOT):** خروجی مالتی‌پلکسر پیش از ورود به جمع کننده، وارد یک زیرمدار (Sub-circuit) می‌شود. اگر بیت فرمان ALU (بیت Add/Sub) برابر با '۱' باشد (عملیات تفریق)، این زیرمدار تمام بیت‌های خروجی مالتی‌پلکسر را NOT می‌کند (مکمل یک). اگر بیت ALU برابر '۰' باشد، داده‌ها بدون تغییر عبور می‌کنند.

۲. **ورودی نقلی (Carry-in):** سیگنال Add/Sub مستقیماً به ورودی C_{in} مدار جمع کننده متصل شده است.

با این طراحی هوشمندانه، در حالت تفریق، عملوند دوم ابتدا NOT شده و سپس با $C_{in} = 1$ جمع می‌شود که دقیقاً معادل محاسبه مکمل دو و انجام عملیات تفریق ($R_0 - Source$) است.





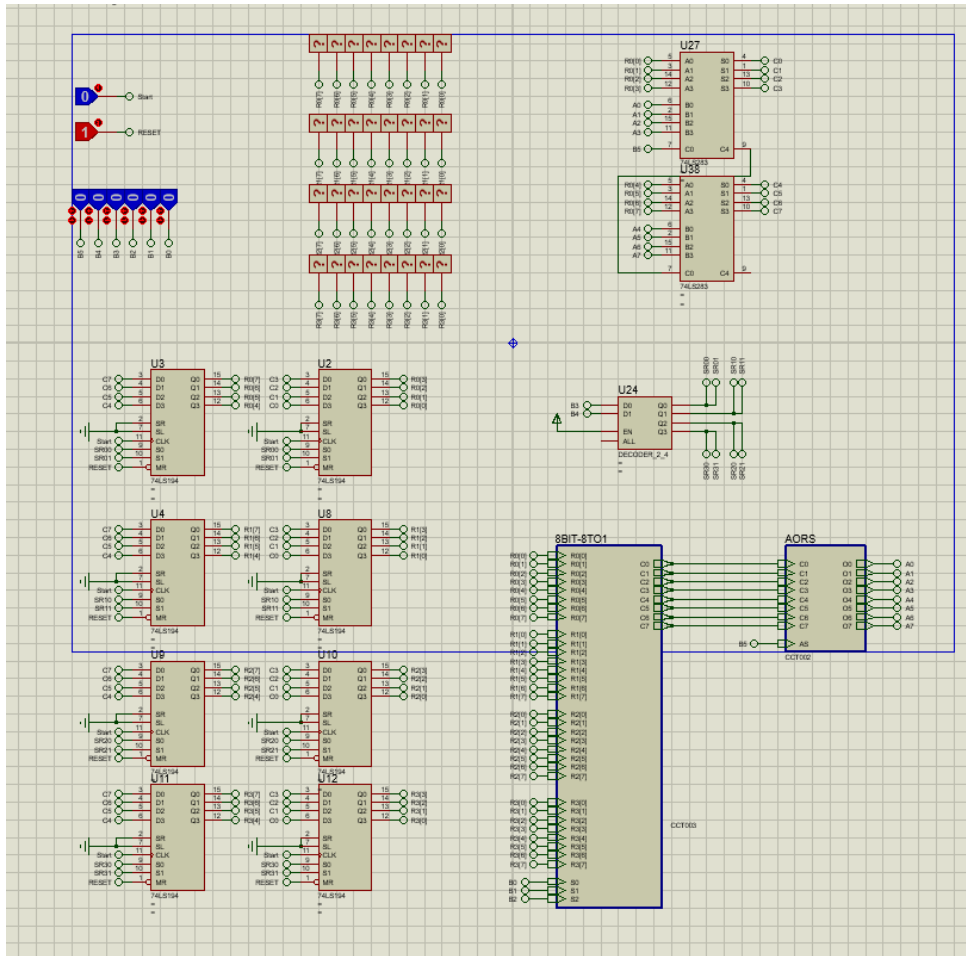
شکل ۳: مدار واحد محاسبات با استفاده از منطق مکمل دو

نتایج کلی شبیه‌سازی

در شکل زیر، نمای کلی و نهایی مدار طراحی شده که شامل یکپارچه‌سازی تمام بخش‌های ذکر شده است، مشاهده می‌شود. با اعمال فرمان‌های شش بیتی مختلف و تغییر وضعیت پالس کلاک به '۱'، صحت عملکرد مدار در انتقال مقادیر بین ثبات‌ها و انجام صحیح عملیات حسابی تأیید گردید.

نتیجه‌گیری

در این آزمایش موفق شدیم با استفاده از قطعات پایه دیجیتال نظیر مالتی‌پلکسرها، دیکدرها، شیفت‌رجیسترها و جمع‌کننده‌ها، یک واحد پردازش ساده با قابلیت اجرای دستورات ماشین طراحی کنیم. استفاده از تکنیک‌هایی مانند استفاده از دیکدر برای مسیریابی سیگنال Load و به کارگیری منطق مکمل دو با تنظیم Carry-in برای یکپارچه کردن مدار جمع و تفریق، باعث بهینگی چشمگیر در تعداد گیت‌ها و اتصالات مدار نهایی شد.



شکل ۴: شماتیک کامل واحد محاسبات و ثبات‌ها

تست، ارزیابی و بررسی حالات مختلف مدار

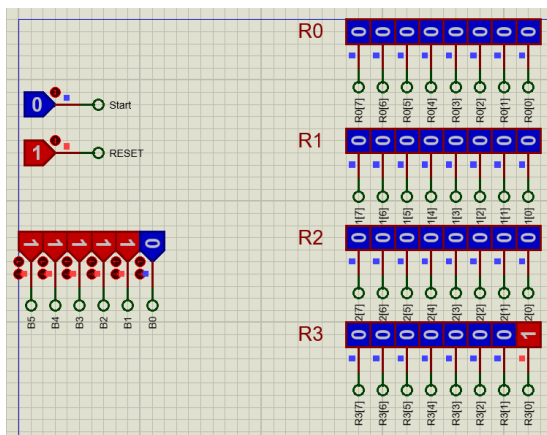
برای صحت‌سنجی عملکرد واحد محاسبه (ALU) سیستم مسیریابی داده‌ها و مکانیزم بارگذاری ثبات‌ها، سناریوهای مختلفی طراحی و روی مدار اعمال شد. این تست‌ها به گونه‌ای انتخاب شده‌اند که تمامی بخش‌های قالب فرمان ۶ بیتی (نوع عملیات، انتخاب مبدأهای مختلف اعم از ثبات و مقدار ثابت، و انواع مقصدها) را به همراه چالش‌های حسابی مانند تولید نتیجه منفی (مکمل دو) ارزیابی کنند. جزئیات حالات آزمایش‌شده و نتایج خروجی ثبات‌ها پس از اعمال پالس کلاک در جدول ۱ خلاصه شده است.

جدول ۱: جدول تست سناریوهای مختلف واحد محاسبه و ثبات‌ها

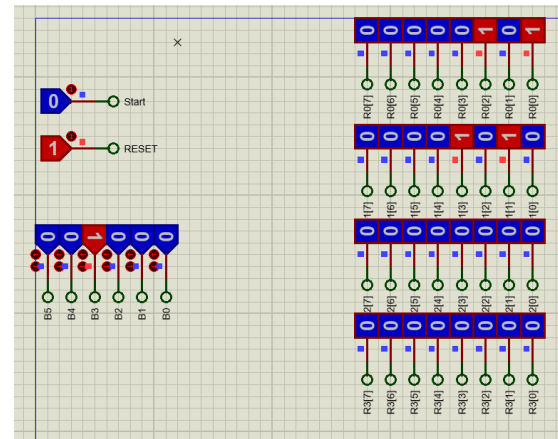
نتیجه نهایی ثبات مقصد	مقصد (Dest)	مبدأ (Source)	نوع عملیات	دستور ۶ بیتی	مقادیر اولیه ثبات‌ها	بف
$R_1 = 5 + 5 = 10$	R_1	R_0	جمع (Add)	0 01 000	$R_0 = 5$	
$R_2 = 12 - 4 = 8$	R_2	R_1	تفریق (Sub)	1 10 001	$R_0 = 12, R_1 = 4$	
$R_0 = 7 + 1 = 8$	R_0	ثابت ۱	جمع (Add)	0 00 101	$R_0 = 7$	
$R_3 = 0 - (-1) = 1$	R_3	ثابت -۱	تفریق (Sub)	1 11 110	$R_0 = 0$	
$R_1 = 3 - 9 = -6$	R_1	R_2	تفریق (Sub)	1 01 010	$R_0 = 3, R_2 = 9$	
$R_3 = 15 + 0 = 15$	R_3	ثابت ۰	جمع (Add)	0 11 100	$R_0 = 15$	

تحلیل فنی نتایج تست‌ها

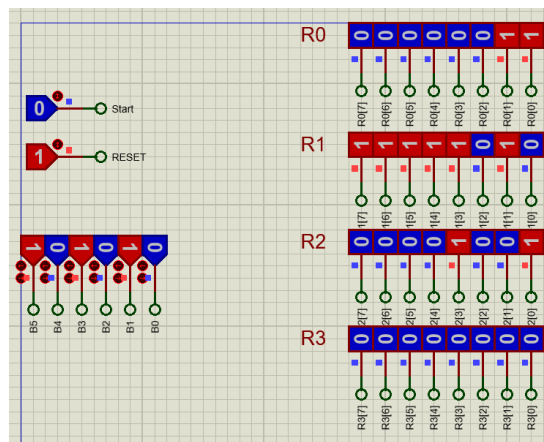
- **بررسی عملگر جمع و صحت دیکدر (تست ۱ و ۳):** در تست اول، دستور 001000 اجرا شد. بیت ارزش بالا، ' (عملیات جمع)، دو بیت مقصد '۰۱' (R_1) و سه بیت مبدأ '۰۰۰' (R_0) هستند. با اعمال کلاک، خروجی دیکدر پایه Load ثبات R_1 را فعال کرد و حاصل جمع ($10 = 5 + 5$) با موفقیت درون آن بارگذاری شد. در تست ۳ نیز افزایش واحدی ثبات R_0 با استفاده از مقدار ثابت ۱ (کانال '۱۰۱' مالتی پلکسر) به درستی ارزیابی گردید.
- **بررسی مقادیر ثابت و منطق تفریق (تست ۴):** یکی از حالات خاص، تفریق مقدار ثابت ۱- از صفر بود. مقدار ۱- در سیستم ۲ بیتی / ۸ بیتی به صورت باینری تکیه بر همه بیت‌های یک (11111111) دارد. با توجه به اعمال دستور تفریق، این مقدار ابتدا در زیرمدار NOT کاملاً معکوس شده (00000000) و با بیت $C_{in} = 1$ جمع شد که معادل مکمل دو (عدد ۱+) است. حاصل جمع آن با $R_0 = 0$ عدد ۱ را به دست داد که صحت منطق تفریق را تایید می‌کند.
- **تولید خروجی منفی و بررسی مکمل دو (تست ۵):** در تست شماره ۵، عملیات تفریق ۹ - ۳ انجام شد که حاصل ریاضی آن ۶- است. از آنجا که سیستم از منطق اعداد علامت‌دار استفاده می‌کند، خروجی به صورت مکمل دو نمایش داده می‌شود. عدد ۶- در فرمت ۸ بیتی برابر با قالب باینری 11111010 (یا FA در مبنای ۱۶) است. در شبیه‌سازی، این وضعیت با روشن شدن ال‌ای‌دی‌های متناظر در خروجی ثبات مقصد (R_1) دقیقاً مطابقت داشت.



(ب) تست چهارم



(آ) تست اول



(ج) تست پنجم

شکل ۵: نمونه‌ای از وضعیت سیگنال‌ها و ثبات‌ها در زمان اجرای تست‌های حسابی

۱.۰ اجرای متوالی دستورات (برنامه نمونه)

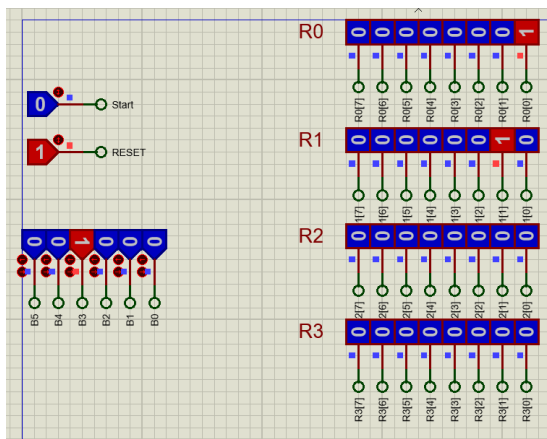
علاوه بر تست‌های مستقل، برای بررسی صحت عملکرد سیستم در ذخیره‌سازی داده‌ها طی کلاک‌های متوالی و استفاده از نتایج قبلی، یک سناریوی اجرای پیوسته (شبیه به اجرای یک برنامه واقعی) روی مدار اعمال شد. در این سناریو فرض بر این است که در حالت اولیه (کلاک ۰) مقدار تمامی ثبات‌ها صفر است. در جدول ۲، مسیر اجرای ۴ دستور پشت سر هم و تغییرات محتوای هر ۴ ثبات پس از لبه بالارونده هر کلاک نشان داده شده است.

جدول ۲: ردیابی اجرای متوالی دستورات و تغییرات وضعیت ثبات‌ها

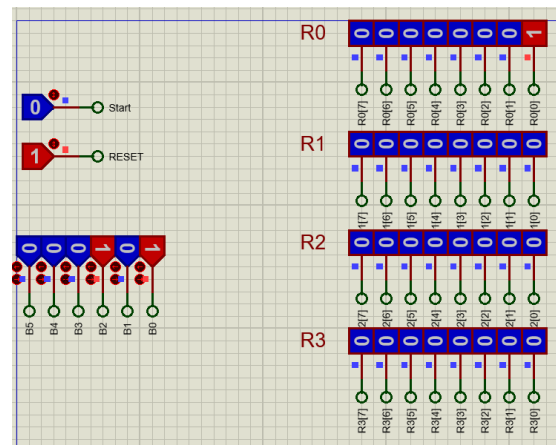
فرمان textbf۶ بیتی	معادل عملیات	مبدأ	مقصد	R°	R¹	R²	R³
-	حالت اولیه (ریست)	-	-	۰	۰	۰	۰
0 00 101	$R^0 \leftarrow R^0 + 1$	ثابت ۱	R^0	۱	۰	۰	۰
0 01 000	$R^1 \leftarrow R^0 + R^0$	R^0	R^1	۱	۱	۰	۰
1 10 001	$R^2 \leftarrow R^0 - R^1$	R^1	R^2	۱	۱	۰	۰
0 11 010	$R^3 \leftarrow R^0 + R^2$	R^2	R^3	۱	۱	۱	۰

تحلیل مسیر اجرا:

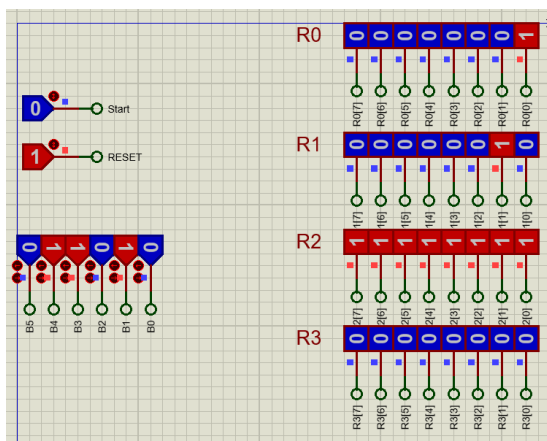
- **کلاک ۱:** با استفاده از ورودی ثابت ۱ در مالتی پلکسر، مقدار ثابت R_0 یک واحد افزایش یافت ($R_0 = 1$).
- **کلاک ۲:** عملوند دوم برابر با خود R_0 انتخاب شد تا مقدار $1 + 1$ محاسبه شده و در R_1 ذخیره گردد ($R_1 = 2$). این مرحله نشان دهنده صحت خواندن و نوشتن همزمان است.
- **کلاک ۳:** تفریق مقدار R_1 از R_0 انجام شد ($2 - 1$). نتیجه برابر با ۱ است که به صورت مکمل دو (در مبنای هگزادسیمال FF) در ثابت R_2 با موفقیت ذخیره شد.
- **کلاک ۴:** در نهایت، محتوای R_2 (یعنی ۱-) با R_0 (یعنی ۱) جمع شد و حاصل آن صفر به دست آمد که در R_3 ذخیره گردید و صحت استفاده مجدد از اعداد منفی در عملیات جمع را تأیید کرد.



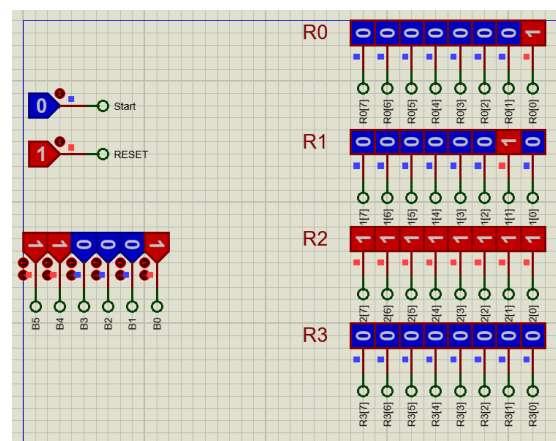
(ب) مرحله دوم



(آ) مرحله اول



(د) مرحله چهارم



(ج) مرحله سوم

شکل ۶: اجرای متوالی دستورات